文章编号:1674-4578(2025)05-0079-04

Git 版本控制系统在软件开发中的应用与实践

赵晓娜

(山西国际商务职业学院,山西 太原 030000)

摘 要:旨在探讨 Git 版本控制系统在软件开发中的应用与实践。首先介绍了 Git 版本控制系统的背景和意义,阐述了研究的目的和内容。然后对 Git 版本控制系统的工作原理、基本应用、GitHub 以及在团队协作的作用进行深入研究和分析。最后,总结了 Git 版本控制系统在软件开发中的重要作用和未来的发展趋势,为软件开发者提供了参考和借鉴。对于推动软件开发效率和质量的提升,具有一定的理论和实践意义。

关键词:Git;版本控制;软件开发;GitHub

中图分类号:TP311.52 文献标识码:A

The Application and Practice of Git Version Control System in Software Development

Zhao Xiaona

(*Shanxi International Business Vocational College*, *Taiyuan Shanxi* 030000, *China*)

Abstract: This paper aims to explore the application and practice of Git version control system in software development. Firstly, the background and significance of the Git version control system are introduced, and the purpose and content of the research are explained. Then it conduct in-depth research and analysis on the working principle, basic applications, GitHub, and its role in team collaboration of the Git version control system. Finally, the important role and future development trends of Git version control system in software development are summarized that providing reference and inspiration for software developers. This paper has certain theoretical and practical significance in promoting the improvement of software development efficiency and quality.

Key words: Git; version control; software development; GitHub

0 引言

在当今软件开发领域,版本控制系统是至关重要的工具之一,它对于管理和追踪软件开发过程中的代码变更、协作和版本发布具有不可替代的作用。随着软件开发项目的规模和复杂度不断增加,传统的代码管理方式已经无法满足需求,因此,引入先进的版本控制系统成为了必然选择。本论文旨在深入探讨 Git 版本控制系统在软件开发中的应用与实践,从而全面了解 Git 的工作原理、基本应用以及对软件开发效率和质量的影响。

1 Git 版本控制系统简介

1.1 Git 的概述和基本原理

Git 是一种分布式版本控制系统,广泛应用于软件开发中。它由 Linus Torvalds 于 2005 年创建,旨在为 Linux 内核开发提供高效的版本管理工具。与传统的集中式版本控制系统不同,Git 将整个项目的代码仓库复制到每个开发者的本地计算机上,使得每个开发者都拥有完整的版本控制功能,并可以独立地进行代码的修改、提交和版本控制,为软件开发提供了强大的支持和便利[1]。

Git 的基本工作原理主要有以下三步:

1) 在工作区中修改文件(新建、修改内容、删除

收稿日期:2024-06-03

作者简介:赵晓娜(1991-),女,山西吕梁人,助教,硕士,研究方向:大数据。

中国知网 https://www.cnki.net

都属于修改操作);

2) 将想要下次提交的更改进行暂存操作;

3) 提交更新,找到暂存区的文件,将快照永久性存储到 Git 仓库。

原理图如图 1 所示。

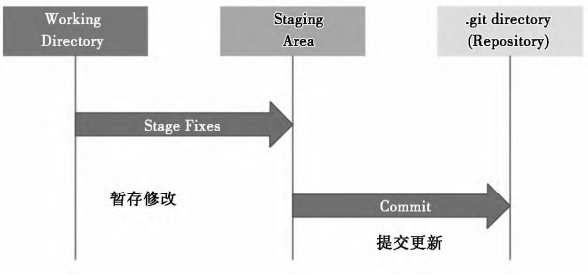


图 1 Git 工作原理图

图 1 中涉及的 Git 有 3 个工作区,分别是工作目录(Working Directory)、暂存区(Staging Area)和仓库(Repository)。其中,工作目录是实际工作的目录,包含了项目的文件;暂存区是一个中间区域,用于存放即将提交到仓库的文件变化;仓库是 Git 的数据库,保存了项目的所有版本历史和元数据信息。

1.2 Git 版本控制系统的优势

1.2.1 集中式版本控制工具 - SVN

在软件开发中,版本控制系统是一项关键的工具,它可以帮助团队有效地管理和追踪代码的变更。除了 Git 之外,还有其它版本控制系统,如集中式版本控制系统 - SVN(Subversion),在此系统中版本库是集中存放在中央服务器的,team 里每个人 work 时从中央服务器下载代码,是必须联网才能工作,局域网或互联网[2]。个人修改然后提交到中央版本库。SVN 结构图如下图 2 所示。

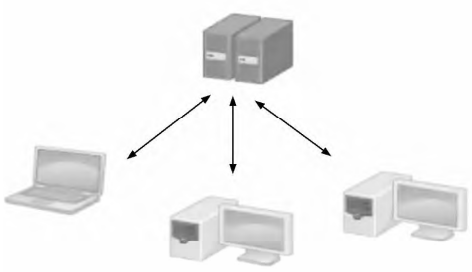


图 2 SVN 结构图

1.2.2 分布式版本控制工具 - Git

Git 相对于 SVN 的主要优势在于其分布式版本控制系统的特性。在 Git 中,每个人的电脑上都拥有一个完整的版本库,无需依赖中央服务器。这意味着在工作时不需要持续联网,因为版本库存储在本地电脑上。多人协作时,只需将各自的修改推送给其他成员,即可互相查看对方的修改,实现高效的协作。由于其具有分布式架构、强大的分支管理、优秀的性能和速度以及高度的数据完整性和安全性等优势,因此在软件开发中得到了广泛的应用和认可。Git 结构图如图 3 所示。

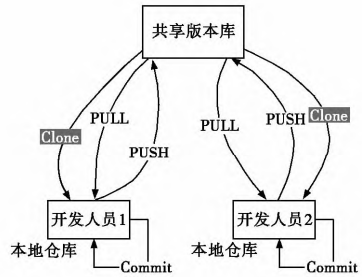


图 3 Git 结构图

2 Git 在软件开发中的基本应用

在软件开发中,Git 作为一种强大的版本控制系统,具有多种基本应用,包括代码版本管理、分支管理和合并、冲突解决以及版本发布与回滚。

2.1 代码版本管理

Git 在软件开发中的代码版本管理方面提供了一系列强大的功能和工具,包括代码提交、版本回溯、版本比较、版本合并、版本标记和历史记录查看等。这些功能使得开发者可以有效地管理和追踪代码的变更,保证软件开发过程的顺利进行[3]。

1) 代码提交(Commit):通过该操作将代码变更保存为一个版本。提交操作可以包括新增、修改或删除文件等操作。

2) 版本回溯(Checkout):即切换到指定的提交版本。这使得开发者可以查看历史提交记录,查找特定版本的代码,并且可以回到之前的某个版本,查看或恢复特定的代码状态。

3) 版本比较(Diff):用于比较不同版本之间的代码差异。通过比较查看两个版本之间的代码修改情况,了解代码的变更内容,并且可以根据比较结果进行相应的操作。

4) 版本合并(Merge):在多人协作开发或者分支开发过程中,开发者可能需要将不同版本的代码进行合并。版本合并(merge)功能,能够智能地将不同版本的代码合并到一起,保持代码的一致性和完整性。

5) 版本标记(Tag):即将特定版本标记为发布版本或者重要版本。标记功能使得开发者可以方便地管理和发布软件版本,以及与团队成员共享重要

中国知网 https://www.cnki.net

的代码状态。

6) 历史记录查看(Log): 即可以查看项目的提交历史记录,了解每次提交的作者、提交时间、提交信息等相关信息,以及具体的代码变更内容。

2.2 分支管理和合并

Git 的分支管理和合并功能使得团队能够更加灵活地进行并行开发、独立功能开发以及版本控制管理。通过分支管理,团队成员可以独立开发和测试不同的功能模块,最终将各个分支的代码合并到一起,保持项目的统一性和完整性。

1) 创建分支(Branch): 创建新的分支来独立开发新功能。创建分支时,可以基于当前分支或者特定的提交版本创建新的分支。

2) 切换分支(Checkout): 切换到不同的分支。即开发者可以在不同的功能或者任务之间进行切换,以便于进行并行开发或者处理不同的代码状态。

3) 合并分支(Merge): 当完成分支开发后,可以使用该操作将分支中的代码合并回主分支(通常是 master 分支)或其它分支中。

4) 删除分支(Branch Deletion): 当分支的任务完成或者不再需要时,删除不需要的分支。删除分支后,分支上的代码将被移除,但对于主分支等其它分支不会产生影响。

2.3 冲突解决

冲突通常发生在多人协作开发或者分支合并的过程中,当两个或多个开发者对同一段代码进行了修改,且这些修改在合并时产生了冲突。Git 的冲突解决功能使得开发者能够有效地处理代码合并过程中可能出现的冲突,保证代码的一致性和完整性[4]。

1) 冲突检测: 当进行分支合并或者拉取远程分支时,Git 会检测是否存在代码冲突。如果两个分支中的同一文件的相同部分被修改,Git 将会标记这些修改为冲突。

2) 冲突标记: Git 会在冲突部分的代码中插入特殊的标记,以标识哪些部分存在冲突。通常,冲突标记会出现在被修改文件中的冲突部分的上方和下方,以便开发者识别。

3) 手动解决冲突: 开发者需要手动解决冲突。这通常包括查看冲突部分的代码,理解两个修改的含义,并决定如何将它们合并为一个兼容的修改。

4) 修改冲突部分: 开发者需要编辑冲突文件,并在冲突标记之间进行修改。

5) 标记冲突解决: 解决冲突后,开发者需要标记冲突为已解决。这通常通过将修改后的文件重新添加到暂存区,并提交合并解决后的版本。

2.4 版本发布与回滚

通过版本发布与回滚,开发团队可以更好地管理软件的发布过程,确保发布的版本质量和稳定性。同时,版本回滚功能也提供了一种应对紧急情况和错误发布的应急措施,帮助团队快速恢复到可用的稳定版本。

1) 版本发布(Version Release): 开发团队在完成一定的功能开发或者修复一些 bug 后,可以选择将代码发布为一个新的版本。通常情况下,版本发布会通过 Git 的标签(tag)功能来实现。开发者可以使用 Git 标记某个特定的提交版本为发布版本,并指定一个版本号或者标识符来标记该版本。

2) 版本回滚(Version Rollback): 在软件开发过程中,可能会出现某个版本存在严重的问题或者不符合预期的情况。为了解决这些问题,开发团队需要快速回退到之前的稳定版本。Git 提供了版本回滚的功能,开发者可以选择回滚到之前的某个提交版本,将代码恢复到之前的状态。

3 Git 在团队协作中的实践 – GitHub

3.1 GitHub 介绍

GitHub 是一个基于 Git 版本控制系统的代码托管平台,于 2008 年创建。它为开发者提供了一个方便的工具和平台,用于管理和共享他们的软件项目。GitHub 的核心功能包括代码仓库托管、版本控制、问题跟踪、代码审查和维基等。用户可以在 GitHub 上创建公共或私有的代码仓库,并通过 Git 进行版本控制,这使得团队成员可以轻松地共同开发项目,并追踪代码的变化历史。GitHub 不仅是一个代码托管平台,更是一个促进开发者交流、学习和合作的社区,推动了软件开发文化的发展[5]。

3.2 GitHub 协同开发

使用 GitHub 可以远程管理项目代码,实现协同开发。在 GitHub 上实现协同开发,一般有以下三种主要形式:

1) Fork 方式:

◆这种方式在大型开源项目中尤为常见。开发

中国知网　https://www.cnki.net

者首先 fork(分叉)一个仓库,从而生成一个与原仓库独立的新分支。在这个分支上,开发者可以自由地进行代码的修改和添加。

◆当开发者完成一定阶段的开发工作后,可以将自己的分支与原仓库进行合并请求(Pull Request)。原仓库的维护者会审查这些请求,根据代码的质量和符合项目的标准与否,决定是否将这些改动合并到主分支。

◆这种方式的优点在于,它允许开发者在不影响主项目的情况下,自由地尝试新的功能和改动。但同时,这也需要维护者花费一定的时间来审查和管理这些合并请求。

2) 组织协作:

◆GitHub 允许用户创建组织,并在组织下管理多个仓库。组织的所有者可以为不同的仓库设置不同的访问权限,从而控制团队成员对代码的访问和修改权限。

◆在组织内部,可以建立团队,并为团队分配特定的仓库权限。这样,团队成员就可以根据他们的角色和职责,在指定的仓库上进行协同开发。

◆通过组织协作的方式,可以更好地管理团队成员的权限和角色,确保项目的安全和顺利进行。

3) 直接协作:

◆仓库的所有者可以直接邀请其他用户作为协作者(Collaborator),赋予他们对该仓库的读写权限。

◆协作者可以在仓库上直接进行代码的提交和修改,与其他协作者共同推进项目的开发。

◆这种方式适用于小型团队或项目,其中团队成员之间有着高度的信任和紧密的协作关系。

无论采用哪种形式,GitHub 都提供了强大的版本控制和代码审查功能,以确保多人协同开发时的代码质量和一致性。同时,通过 GitHub 的 Issue 跟踪和讨论功能,团队成员还可以有效地沟通和协作,共同解决项目中的问题和挑战。

4 结论与展望

随着软件开发的不断演进,Git 作为分布式版本控制系统将在未来持续发展。其发展趋势包括更强大的协作支持、提升性能和扩展性、与新兴技术的整合以及跨行业的应用拓展。Git 将继续为团队协作提供更灵活的工作流程支持,同时不断优化性能以适应日益复杂的项目需求。此外,与人工智能、区块链等技术的整合将为 Git 带来新的应用场景,进一步拓展其在软件开发以及其它行业中的应用前景。

参考文献

[1] 王真. 版本控制工具在软件开发项目管理中的应用: 以 GIT 为例[J]. 项目管理技术,2020,18(06):131-134.
[2] 秦佳. 软件配置管理中版本控制的研究[J]. 软件,2019,40(03):137-139.
[3] 戴科峰,仇礼钦,盛飞龙,等. 碳化硅外延设备控制系统的设计与实现[J]. 机电工程技术,2022,51(09):132-136.
[4] 刘爱群,于健,王瑞,等. 基于 Git 技术的机车车载应用软件集中控制系统研究[J]. 铁道运输与经济,2022,44(02):52-58.
[5] 李文竞,王丽. 国内功能对等理论研究状况的可视化分析[J]. 郑州航空工业管理学院学报(社会科学版),2020(02):68-77.

(上接第 72 页)

参考文献

[1] 中共中央办公厅. 国务院办公厅印发《关于进一步加强非物质文化遗产保护工作的意见》[J]. 中华人民共和国国务院公报,2021(24):14-17.
[2] 邹颖. 中国非物质文化遗产中手工刺绣的传承与发展[D]. 南昌: 南昌大学,2021.
[3] 张宇霞. 民间染织刺绣纹样在平面设计中的应用分析—以苏绣为例[J]. 纺织报告,2023,42(12):54-56.
[4] 虞倩雯,王建萍. 苗绣数字化保护现状与传承路径探究[J]. 西部皮革,2024,46(05):137-139.
[5] 高照. 基于 VR 全景技术的江苏段大运河虚拟漫游系统设计与实现[J]. 新媒体研究,2023,9(02):11-14,19.

中国知网 https://www.cnki.net